

CS & IT ENGINEERING



Data Structure & Programming

Stack and Queues

Lecture No.- 01



By- Satya sir

Recap of Previous Lecture



- Doubly Linked Lists

- Operations

- Circular Linked List

- Operations



Topics to be Covered



- PYQ Practice
- Stack
 - Nature
 - Operations
 - Permutations



Topic : Linked Lists

Time Complexity chart



operation	<u>Insertion</u>						<u>Deletion</u>						Search/ Access/ Traverse
	<u>Beginning</u>		<u>middle</u>		<u>End</u>		<u>First Node</u>		<u>middle Node</u>		<u>Last Node</u>		
	Without Tail	With Tail	Without Tail	With Tail	Without Tail	With Tail	W.O Tail	With Tail	W.O Tail	With Tail	W.O Tail	With Tail	
Linked List													
Singly Linked List	$O(1)$	$O(1)$	$O(n)$	$O(n)$	$O(n)$	$O(1)$	$O(1)$	$O(1)$	$O(n)$	$O(n)$	$O(n)$	$O(n)$	$O(n)$
Doubly Linked List	$O(1)$	$O(1)$	$O(n)$	$O(n)$	$O(n)$	$O(1)$	$O(1)$	$O(1)$	$O(n)$	$O(n)$	$O(n)$	$O(1)$	$O(n)$
Circular Singly Linked list	$O(n)$	$O(1)$	$O(n)$	$O(n)$	$O(n)$	$O(1)$	$O(n)$	$O(1)$	$O(n)$	$O(n)$	$O(n)$	$O(n)$	$O(n)$
Circular Doubly Linked list	$O(1)$	$O(1)$	$O(n)$	$O(n)$	$O(1)$	$O(1)$	<u>$O(1)$</u>	<u>$O(1)$</u>	$O(n)$	$O(n)$	<u>$O(1)$</u>	<u>$O(1)$</u>	



Topic : Linked Lists

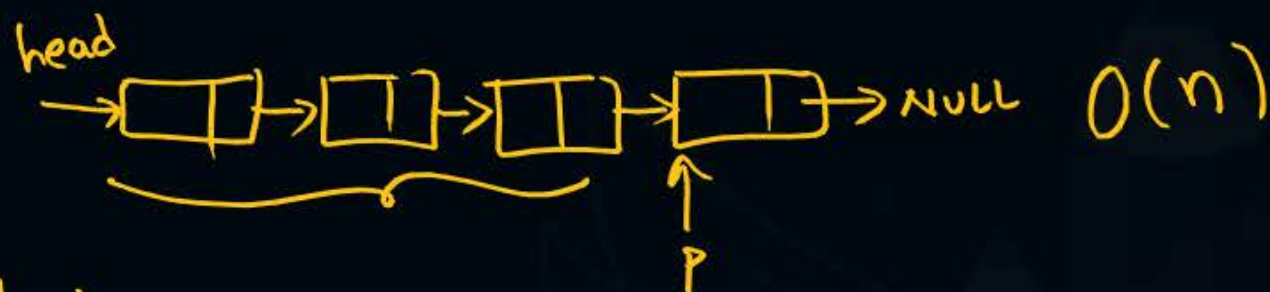


#Q. Let SLLdel be a function that deletes a node in a singly-linked list given a pointer to the node and a pointer to the head of the list. Similarly, let DLLdel be another function that deletes a node in a doubly-linked list given a pointer to the node and a pointer to the head of the list.

Let n denote the number of nodes in each of the linked lists. Which one of the following choices is TRUE about the worst-case time complexity of SLLdel and DLLdel?

- A. SLLdel is $O(1)$ and DLLdel is $O(n)$
- B. Both SLLdel and DLLdel are $O(\log(n))$
- C. Both SLLdel and DLLdel are $O(1)$
- ☒ D. SLLdel is $O(n)$ and DLLdel is $O(1)$

Deletion with Tail (Last Node)



GATE 2023



Topic : Linked Lists



What is the worst case time complexity of inserting n elements into an empty linked list, if the linked list needs to be maintained in sorted order ?

- (A) $\Theta(n)$
- (B) $\Theta(n \log n)$
- ☒ (C) $\Theta(n^2)$
- (D) $\Theta(1)$

Insertion Sequence: 70, 60, 50, 40, 30, 20, 10 $\Rightarrow$ Insertion at Beginning



$O(1)$ for each Element
 $\Rightarrow$ for n Elements : $O(n)$

GATE 2020

Insertion Sequence: 10, 20, 30, 40, 50, 60, 70 : Insertion at End



$$1 + 2 + 3 + 4 + \dots + n = \frac{n(n+1)}{2} = \frac{n^2 + n}{2} \Rightarrow O(n^2)$$

Insertion Sequence: 30, 70, 20, 10, 50, 40, 60



Insertion at Random Position $\Rightarrow O(n^2)$



Topic : Linked Lists



The following C function takes a single-linked list of integers as a parameter and rearranges the elements of the list. The function is called with the list containing the integers 1, 2, 3, 4, 5, 6, 7 in the given order. What will be the contents of the list after the function completes execution?

```
void rearrange(struct node *list)
{
    struct node *p, *q;
    int temp;
    if ((!list) || !list->next)
        return;
    p = list;
    q = list->next;
    while(q)
    {
        temp = p->value;
        p->value = q->value;
        q->value = temp;
        p = q->next;
        q = p?p->next:0;
    }
}
```

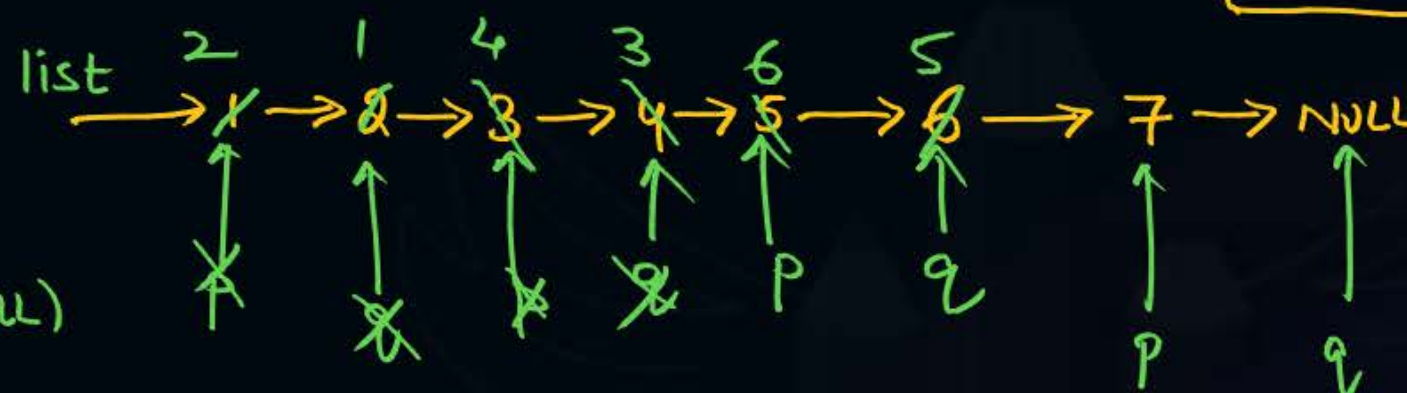
(A) 1,2,3,4,5,6,7

☒ (B) 2,1,4,3,6,5,7

(C) 1,3,2,5,4,7,6

(D) 2,3,4,5,6,7,1

GATE 2008



temp
2 3 5

SLL



Topic : Linked Lists



The following C function takes a singly-linked list as input argument. It modifies the list by moving the last element to the front of the list and returns the modified list. Some part of the code is left blank.

```
typedef struct node
{
    int value;
    struct node *next;
}Node;

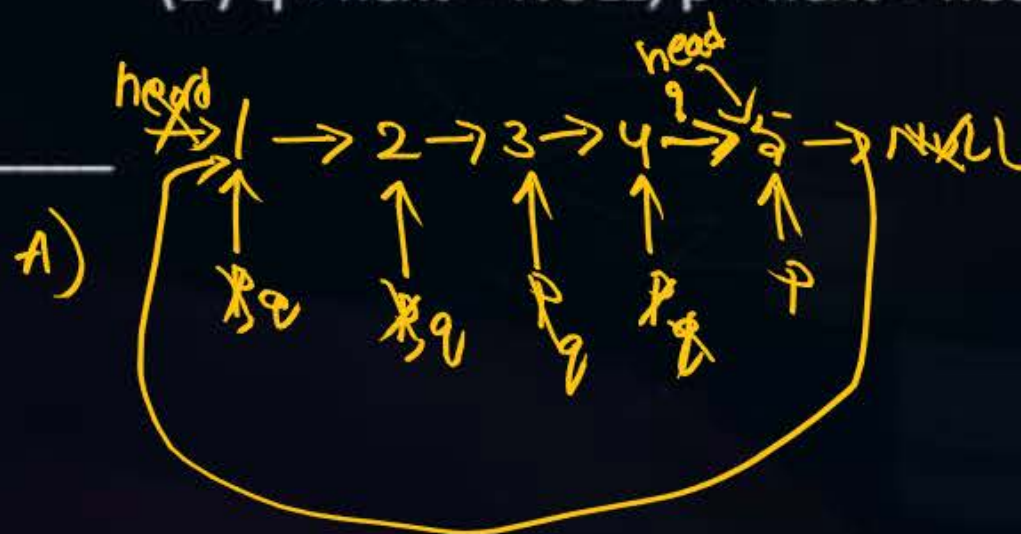
Node *move_to_front(Node *head)
{
    Node *p, *q;
    if ((head == NULL) || (head->next == NULL))
        return head;
    q = NULL; p = head;
    while (p->next != NULL)
    {
        q = p;
        p = p->next;
    }
    _____
    return head;
}
```

q/p
head
→ 1 → 2 → 3 → 4 → 5 → NULL

o/p:
head
→ 5 → 1 → 2 → 3 → 4 → NULL

Choose the correct alternative to replace the blank line.

- ☒ (A) q = NULL; p->next = head; head = p;
- ☐ (B) q->next = NULL; head = p; p->next = head;
- ☐ (C) head = p; p->next = q; q->next = NULL;
- ☐ (D) q->next = NULL; p->next = head; head = p;



GATE 2010



Topic : Linked Lists



```
Node *move_to_front(Node *head)
{
    Node *p, *q;
    if ((head == NULL) || (head->next == NULL))
        return head;
    q = NULL; p = head;
    while (p->next != NULL)
    {
        q = p;
        p = p->next;
    }
    _____
    return head;
}
```

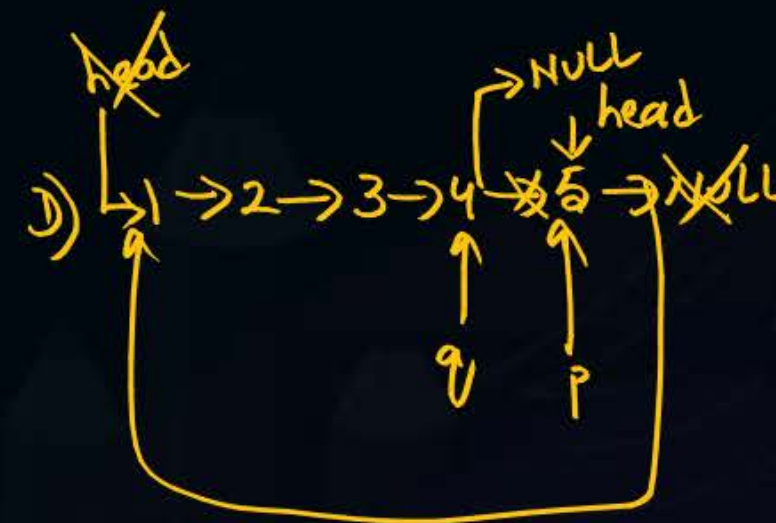
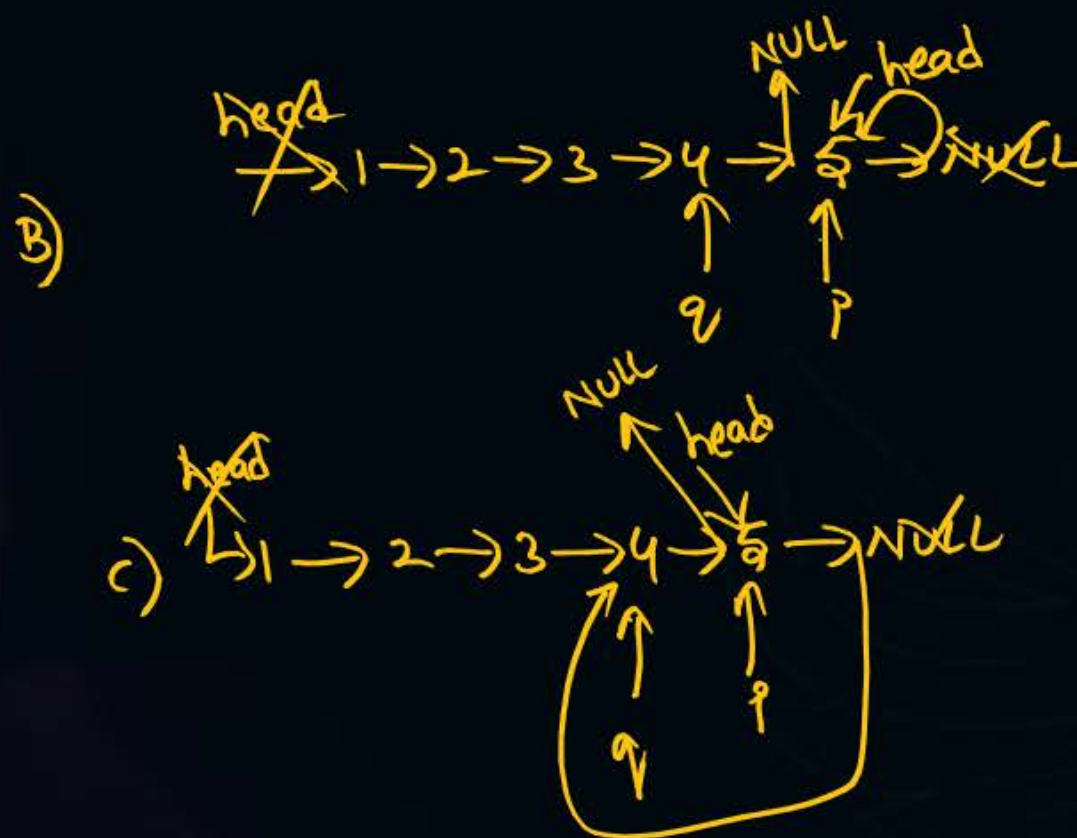
Choose the correct alternative to replace the blank line.

~~(A)~~ q = NULL; p->next = head; head = p;

~~(B)~~ q->next = NULL; head = p; p->next = head;

~~(C)~~ head = p; p->next = q; q->next = NULL;

~~(D)~~ q->next = NULL; p->next = head; head = p;





Topic : Linked Lists



H/W

Consider the function f defined below.

```
struct item
{
    int data;
    struct item * next;
};
int f(struct item *p)
{
    return ((p == NULL) || (p->next == NULL) ||
            ((p->data <= p->next->data) &&
             f(p->next)));
}
```

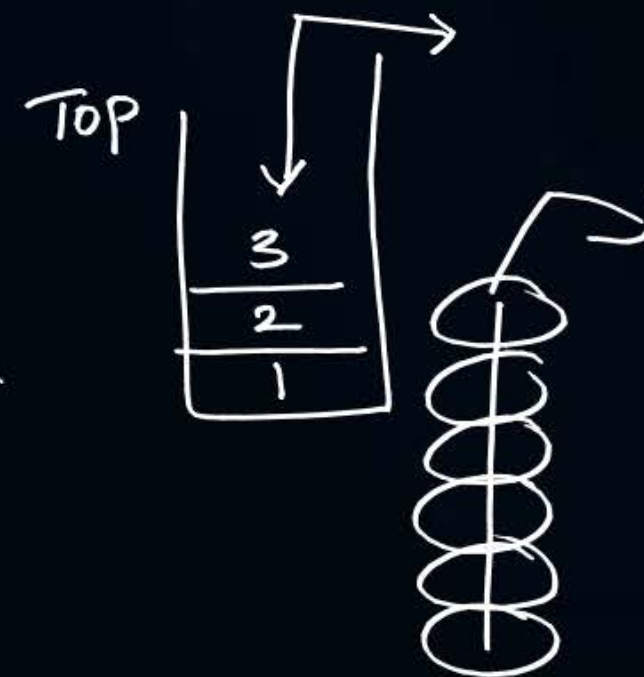
For a given linked list p, the function f returns 1 if and only if

- (A) the list is empty or has exactly one element
- (B) the elements in the list are sorted in non-decreasing order of data value
- (C) the elements in the list are sorted in non-increasing order of data value
- (D) not all elements in the list have the same data value.



STACK

- It is Linear Data structure
- It is a list which allows Insertion and deletion from only one end.
- It is a Last-In-First-Out (LIFO) data structure
- Insertion — Push operation
- Deletion — POP operation
- Access — Peek operation



Ex: Consider an Empty stack S

Push(value, S): Insert Value into Stack.

Pop(S): Deletes the last inserted / top Element and return Element deleted.

Peek(S): Return the top Element without Deletion.



Topic : Stack Operations, Permutations



Ex: Consider an empty stack S . Now Perform below Operations in the given order:

1) Push(10, S)

2) Push(20, S)

3) Push(30, S)

4) $a = \text{Peek}(S)$ // $a = 30$

5) Push(40, S)

6) $b = \text{Pop}(S)$ // $b = 40$

7) $c = \text{Peek}(S)$ // $c = 30$

8) Pop(S)

9) Push(50, S)

10) $d = \text{Pop}(S)$ // $d = 50$

$$a+b+c+d = \frac{150}{30+40+30+50}$$

50
40
30
20
10



Topic : Stack Operations, Permutations



Ex:2 Consider 2 empty stacks S_1, S_2 . Now Perform below operations in the given order:

1) Push(5, S_1) ✓

2) Push(6, S_2) ✓

3) Push(Peek(S_1), S_2) ✓

4) Pop(S_1) ✓

5) Push(Pop(S_2), S_1) ✓

6) Push(9, S_1) ✓

7) Push(10, S_2) ✓

8) Push(Peek(S_2), S_1) ✓

9) Push(Pop(S_2), S_1) ✓

10) Pop(S_1)

11) Push(Peek(S_1), S_2)

12) Push(12, S_1)

13) Push(Pop(S_1), S_2)

14) $a = \text{Peek}(S_2) \neq a = 12$

15) $b = \text{Pop}(S_1) \neq b = 10$

$$a + b = \underline{22}$$

12
10
10
9
5
5

S_1

12
10
10
5
6

S_2



2 mins Summary



- LL Time Complexities
- PYQ Practice
- Stack Operations

To be Contd...



THANK - YOU